

Project Architecture and Logic Guide

Embedded Security – SSH Tunneling for Modbus/TCP

This document outlines the core logic, structural decisions, and key workarounds implemented throughout the Modbus/TCP SSH Tunneling project. It serves as a technical companion to explain *why* certain design choices were made.

1. Key Concepts

The Vulnerability of Modbus/TCP

Modbus is a legacy industrial protocol designed for isolated serial networks. When adapted for modern TCP/IP networks, it completely lacks built-in security mechanisms such as encryption or authentication. All function codes and register values are transmitted in plaintext, leaving industrial infrastructure highly susceptible to eavesdropping and manipulation.

The SSH Tunnel Solution

Instead of rewriting the fundamental Modbus protocol, this project implements a "wrapper" security approach. By utilizing SSH Local Port Forwarding, Modbus traffic is intercepted, encrypted, and routed through a secure shell before reaching the destination server.

2. Structural Logic & Design Decisions

The "Translator" Concept (Port 2222 vs Port 22)

A common point of confusion is why the Modbus client connects to Port 2222 instead of the standard SSH Port 22.

- **The Language Barrier:** The SSH server listening on Port 22 only understands the SSH protocol. If a Modbus client connects directly to Port 22, the connection will instantly crash due to protocol mismatch.
- **The Doorway:** By creating a local tunnel entrance on Port 2222, the SSH software acts as a translator. The Modbus client speaks its native protocol to Port 2222. The tunnel intercepts this data, wraps it in an SSH envelope, and securely transmits it to Port 22.

Client-Side Data Generation

In a standard SCADA architecture, the Server (PLC) generates sensor data, and the Client (HMI) simply reads it. In this project, the logic is reversed: the Client actively generates simulated **Boiler Temperature (75-80°C)** and **Motor Speed (1450-1500 RPM)** and writes it to the Server.

- **The Logic:** If the Server passively generated data, packet sniffing would only demonstrate the encryption of Read commands. By forcing the Client to actively push data, we successfully prove that **both Write and Read operations are fully secured** by the tunnel, making for a much stronger security demonstration.
-

3. Technical Workarounds

The Python Tunnel Workaround

The project provides two methods for tunneling: Native OS SSH (make tunnel-native) and a Python-based tunnel (make tunnel-python).

- **The Problem:** The Native OS tunnel relies on the operating system's built-in SSH server. On Windows machines, the OpenSSH server is often disabled by default, causing the native tunnel test to fail.
- **The Workaround:** The `modbus_tunnel.py` script utilizes the `sshtunnel` Python library. Because it runs entirely within the Python interpreter, it bypasses native OS limitations, ensuring the project is 100% cross-platform (macOS, Linux, Windows).

The Benchmark "Food Critic" Workaround

To test the feasibility of the encryption overhead on embedded systems, we built an automated testing script (`modbus_benchmark.py`).

- **The Logic:** Instead of requiring the user to manually run the client thousands of times, the benchmark script temporarily replaces the standard client. It acts as a high-speed "food critic," directly importing the Modbus library to fire 1,000 rapid requests. This allows it to mathematically calculate exact millisecond latency and CPU percentage overhead across Plaintext, Native, and Python routing scenarios.